

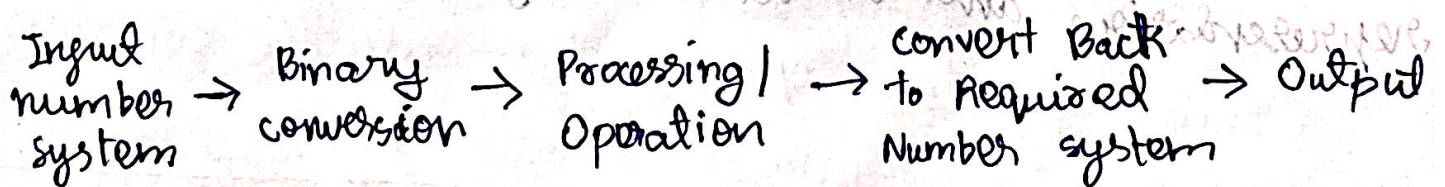
1. * Fractional decimal numbers may produce long or repeating binary values, causing truncation errors.
- * Incorrect grouping of binary digits into 3-bit sets leads to wrong octal conversion
- * Any error in binary representation directly changes the final octal output, reducing transmission accuracy

2. a. * Hexadecimal is better because it represents binary numbers in shorter and more compact form
- * It provides easy binary conversion, reducing processing overhead and improving readability.

3. * Correct sequence : Encoder $\rightarrow$ Comparator $\rightarrow$ Decoder
- * The encoder converts input signal into coded form, the comparator checks / compares them.
- * And the decoder converts the result back to readable output.
- * Improper sequencing cause incorrect signal processing leading to wrong comparisons and inaccurate final outputs.

4. * The appropriate encoder is a Priority Encoder because it can handle multiple active inputs at the same time
- * It gives priority to the highest-priority input and produces a correct binary outputs, avoiding ambiguity and incorrect processing

5. * correct sequence:



- * Binary conversion should happen before processing because digital systems operate internally in binary form.
- * incorrect sequencing can cause wrong calculations, data loss and inaccurate final output due to improper conversion handling.

6. * The error is that the program processes only the integer part of the binary number and ignores the fractional bits after the binary ~~input~~ point.

- * As a result, the decimal output becomes incomplete and inaccurate for fractional binary values.

7. * This issue occurs because higher-order bits are not checked correctly and cascading connections between comparators are faulty.

- * Due to this, the circuit may ignore significant bit differences and wrongly indicate equality for unequal inputs.
- * This reduce comparison accuracy and can lead to incorrect decision-making in the system.

8. * The fault is incorrect decoder logic, causing multiple outputs to become active for a single input instead of only one valid output.

- * This leads to display errors, incorrect data representation and unreliable system operation.

9. * Use the 2's complement representation model for signed numbers in the digital system

* In this method, the MSB (Most significant bit) represents the sign:

0 $\rightarrow$ positive

1 $\rightarrow$ negative

* Negative numbers are represented using 2's complement, allowing addition and subtraction to be performed with same circuitry

* It avoids separate sign handling and reduces hardware complexity

* This model is accurate, because it provides accurate arithmetic operations, simple implementation and consistent processing in digital systems.

10. * Multiplexer (MUX) ~~represents~~ reduces multiple input lines into one output, saving wiring and improved data transmission efficiency.

* Demultiplexer (DMUX) distributes one input to multiple outputs, useful for controlled data delivery.

* Using only MUX ~~represents~~ increases routing efficiency but limits output distribution flexibility

* Using only DEMUX supports distribution but requires more transmission paths

* The optimal approach is to combine MUX and DEMUX together, enabling ~~scalable~~ scalable, efficient and flexible data routing with reduced hardware complexity.

11. * Represent the numbers in 4-bit 2's complement:

$$-4 = (1100)_2$$

$$-2 = (1110)_2$$

* Add these binary values:

$$\begin{array}{r} 1100 \text{ (4)} \\ 1110 \\ \hline 11010 \end{array}$$

* Ignore the carry bit outside 4 bits, so the result becomes 1010.

* 1010 in 2's complement represent -6

* Since the 4-bit range is -8 to +7, the result -6 is valid and no overflow

12. * A 4:1 multiplexer has inputs I_0, I_1, I_2, I_3 , select lines S_1, S_0 and one output Y .

* The output depend on the select lines:

$$00 \rightarrow I_0$$

$$01 \rightarrow I_1$$

$$10 \rightarrow I_2$$

$$11 \rightarrow I_3$$

* Boolean expression of the 4:1 MUX is

$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

* Each term enables only one input according to the select line combination.

* Thus, the logic ensures that only the selected input appears at the output, providing correct data selection.

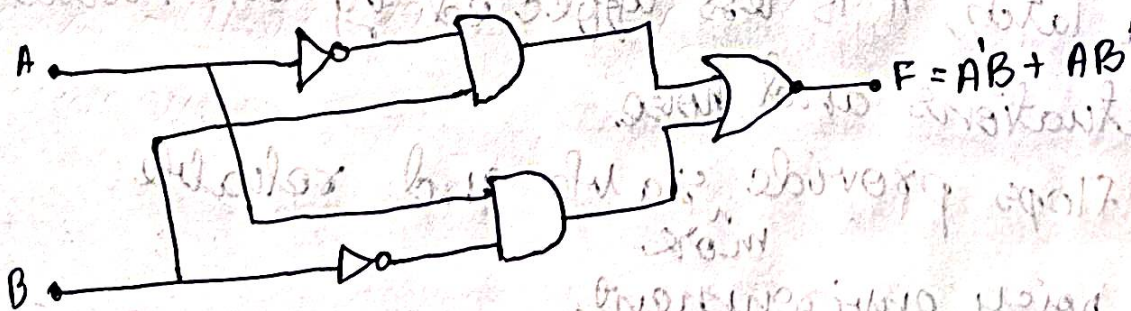
13. * Basic gates give flexibility but make large circuit complex
* standard combinational modules simplify circuit design and implementation.
* Using modules reduces wiring, power usage and design time
* Modules improves scalability, reliability and maintainance
* Best approach: use ^{standard} combinational modules for efficient and scalable circuit design.

14. * simple ALU designs are easy to implement but provide limited performance and scalability
* complex ALU architectures support faster and multiple operations but increase hardware cost and power consumption
* parallel processing improves speed but requires more circuitry
* Modular ALU design allows easy expansion and maintainance
* The optimal design is a modular and scalable ALU architecture that balances speed

15. $F = A'B + AB'$

* Implement using:

- 2 NOT gates for A' and B'
- 2 AND gate for $A'B$ and AB'
- 1 OR gate to combine outputs



* Fourth table

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

* The output is 1 only when inputs are different

* Hence, the function behaves as an XOR gate.

16. * Design the ALU with separate Arithmetic Unit and Logic unit connected through control signals.

* The Arithmetic Unit performs operations like addition, subtraction, increment and decrement.

* The logic unit performs AND, OR, NOT and XOR operations.

* A control unit selects the required operation, based on instruction signals and sends the result to the output.

* This model is effective because it provides fast processing, flexible operations and scalable processor design.

17. * A flip flop stores data only at specific clock intervals, providing controlled data ~~signal~~ storage.

* Unlike a latch, it is less affected by continuous input fluctuations and noise.

* $\therefore$ flip flops provide stable and reliable ^{more} operation in noisy environment.

18. * A D flip flop is clock-controlled, so it stores the input data only when a clock pulse occurs.
* Continuous input changes do not affect the output until the next clock edge, ensuring stable and controlled operations.

19. * The problem occurs in an asynchronous (ripple) counter due to propagation delay between flip flops.
* Delay cause incorrect or unstable output during counting.

* Using a synchronous counter reduces delay by clocking all flip flops together

20. * A 4-bit binary counter can represent $2^4 = 16$ states.
* Therefore, the maximum count values before reset is $(15)_{10}$ or $(1111)_2$

21. * Latches change output whenever input changes, so noise cause instability.
* Unwanted signal changes may produce incorrect outputs in a latch.
* Flip Flops change only at clock edges, giving stable and reliable operation.

22. * When $S=1$ and $R=1$ simultaneously in an SR latch, it produces an invalid (forbidden) condition because both outputs become unstable or unpredictable.

23. * A shift register converts parallel data into serial data by shifting one bit at a time.

- * Each clock pulse moves the bit sequentially through the register for transmission over a single line.
- * This reduces the number of communication lines required, making data transmission efficient and economical.

24. initial data = 1011
After one right shift with 0 inserted at the left
output = 0101

25. *
- * Clock signal synchronizes all flip-flops to operate together
 - * output change only at clock ~~speed~~ pulses, avoiding unpredictable changes.
 - * It prevents timing error caused by asynchronous input
 - * Synchronized operation improves coordination between circuit stages.
 - * Proper clock frequency ensures both speed and reliable performance

26. *
- * Registers temporarily store binary data in digital system for processing.
 - * The value 1101 is stored in ~~the~~ 4-bit register and placed on the common bus during transfer.
 - * control signals enable the source register to send data and destination register to receive it
 - * The clock signal synchronizes the transfer so data moves at the correct instant
 - * Proper timing and control prevent data loss, ~~any~~ corruption and transfer errors.

27. * Edge-triggered flip-flops capture data only at the rising edge of the clock signal.
- * Input changes between clock pulses do not affect the stored data
 - * This provides stable and reliable data storage in digital circuit.
 - * Level-triggered circuits may continuously respond to input changes during the clock level, causing instability
 - * Hence, edge triggering reduces timing errors and improves synchronization.

28. * Initial content: Register A = 101, Register B = 000
- * After the control signal and next clock pulse, Register B becomes 101
 - * All three bits are transferred simultaneously during the same clock pulse.
 - * Control signals enable Register A to send data and Register B to load the data.
 - * The clock synchronizes retrieval and storage, ensuring accurate transfer without data loss.

29. * Registers temporarily store binary data and intermediate results in digital systems.
- * They provide fast access to data required for arithmetic and logical operations.
 - * Registers help the ALU process efficiently during instruction execution.
 - * Clock signals control accurate storing and transfer of data between registers.
 - * This improves system speed, synchronization, and reliable data processing.

30. * Address lines select a specific memory location in RAM.
* The decoder activates only one memory cell for the given address.
* 16 memory locations requires 4 address lines ($2^4 = 16$)
* In read operation, data is taken from the selected location.
* In write operation, data is stored only in the selected location, avoiding corruption.
-

31. * Initial register value = 1010
* Input data applied = 011
* At the next clock pulse, the register loads the new input data.
* Previous data is replaced by the new value.
* $\therefore$ the register contents after the clock pulse = 011
-

32. * RAM temporarily stores data and program instructions required by the processor.
* Memory location in RAM are organised using unique binary address.
* The processor sends an address to select the required memory location.
* The stored data at an address to select the required memory location.
* Efficient addressing and access improves system speed and accurate operation.
-

33. *Fetch - Instruction is fetched from memory into the instruction Register (IR)

*Decode & Execute - Control unit decodes the instruction and ALU performs the operation.

*Store & Update - Result is stored and Program Counter (PC) moves to the next instruction.

*Justification: order execution ensures correct timing, proper data flow and error-free instruction processing.

34. *Execution time is given by:

$$\text{Execution Time} = \text{Instruction count} \times \text{Cycles per Instruction (CPI)} \times \text{Clock Cycle Time}$$

* If the ~~type~~ cycle count (CPI) increases, the processor requires more clock cycles to complete instructions, so the total execution time also increases. Lower CPI reduces processing duration and improves performance.

35. i) Instruction Decode - The control unit reads the opcode and identifies the required arithmetic operation and source register.

ii) Control signal generation - The hardwired control unit generates timing and control signal based on the decoded opcode.

iii) ALU Execution - Control signal activates the ALU, operands are taken from registers and the arithmetic operation is performed at the correct clock timing.

* proper sequencing ensures synchronized execution and prevents timing errors during instruction processing.

36. IF opcode = ADD
Generate ALU-Add signal
ELSE IF opcode = SUB
Generate ALU-Sub signal
ELSE IF opcode = LOAD
Generate Memory-Read signal
ELSE IF opcode = STORE
Generate Memory-Write signal
END IF

37. * More instructions increases control logic complexity and execution management difficulty.
* Hardwired control units are faster, but difficult to expand for new instructions.
* Microprogrammed control units are more flexible and scalable, making system upgrades easier and more efficient.

38. * The control unit controls and synchronizes the operations of memory, register and ALU using control signal.
* This ensures accurate data flow and reliable instruction execution.

39. * Combinational decoding is fast, but needs more hardware and power.
* Microprogrammed decoding uses less hardware and is more flexible, but instruction execution become slower.
* For IoT devices, microprogrammed decoding is more efficient due to lower hardware and energy usage.

40. * specialized decoding separates arithmetic and I/O instructions into different execution paths, reducing unnecessary processing delay.

* This improves execution speed efficiency and makes system operations more organized and clear.

41. * The PC increments normally, so address generation is correct.

* MAR receives the correct address, but MDR sometimes loads old data because memory read occurs before memory ready.

* Due to outdated MDR data, incorrect instructions are fetched and executed.

* Multi-byte instructions worsen the issue because improper alignment cause partial or wrong instruction fetching.

* The primary fault is lack of memory-ready synchronization in the fetch cycle, causing invalid MDR loading and instruction execution errors.

42. * Instruction is fetched from memory and decoded by the control unit.

* operands are read from general-purpose register and transferred to the ALU

* ALU performs arithmetic operation; temporary registers stores intermediate multiplication result.

* result is written back to the destination register and status ~~are~~ flags are updated.

* control unit checks branch condition and updates PC for branch or next instruction using synchronized control signal.

43. * Prefetching reduces instruction fetch delay by loading instructions only.
- * However, prefetching increases hardware complexity, power usage and may fetch unused instructions.
- * Optimized bus organization speeds up MAR-MDR data transfer and memory access.
- * Better Bus designs improves scalability and efficient data movement.
- * Prefetching improves speed, while bus optimization gives better overall efficiency.

44. * Multi-cycle instructions reduces pipeline efficiency by occupying ALU resources for longer time.
- * Branch instructions create control hazards and may cause pipeline stalls, reducing throughput.
- * Temporary registers help store intermediate results and support smooth micro-operation execution.
- * Correct sequencing and hazard handling are necessary to maintain accurate execution and avoid data conflicts.
- * Techniques like pipelining, branch prediction and forwarding improves performance while preserving execution correctness.

45. * Effective address calculation:
- $$EA = \text{Base (4000)} + \text{Index (120)} + \text{Offset (30)} = 4150$$
- computed by ALU under control unit coordination

- * MAR load: The computed effective address (4150) is placed into MAR for memory access.
- * Cache check: System checks cache first. If miss occurs, access is sent to main memory via unified bus.
- * Memory fetch: MDR loads operand from memory, with possible wait states due to memory delay.
- * Instruction completion: Operand is transferred to IR/CPU register, ensuring correct MAR-MDR synchronization throughout the cycle.

46. * Write-back stage bottleneck occurs when multiple instructions complete at the same time.
- * More hardware reduces delay but increases cost and power usage.
 - * Shared paths save resources but may cause contention and stalls.
 - * Priority-based scheduling helps manage simultaneous write-backs efficiently.
 - * Best solution: balanced design with limited hardware and dynamic priority control.

47. * Register addressing gives faster execution since data is directly available in CPU registers, improving performance.
- * But registers are limited, so it reduces scalability for large datasets and complex programs.
 - * Memory addressing supports large data storage, making it more scalable for complex workloads.
 - * However, it is slower due to cache/main memory access delays, reducing throughput.

40. *ALU produces result and sends it to destination register during the write-back stage.
- * control unit activates write-enable signals to store result and simultaneously update condition flags.
 - * Register write and flag update must occur in a controlled sequence to avoid overlap conflicts.
 - * synchronous clock ensures both operations complete before starting the next instruction fetch.
 - * Proper synchronization guarantees correct write-back completion and prevents timing errors in the instruction cycle.